Software Requirements Specification

for

Waterfall Project

Version 1.0 approved

Prepared by Hlina T, Adam B, Arish S, Rahim B

CSCI 4801

October 1st, 2025

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History iii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 1
2.1 Product Perspective 1
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features 4
4.1 Command Line for UI 4
4.2 File Input By Name 5
4.3 Audit File 6
4.4 Display Results 7
4.5 Tie Handling 8
4.6 STV Algorithm 10
4.7 Plurality Algorithm 11
4.8 Shuffle 13
4.9 Help Window 14
5. Other Nonfunctional Requirements 15
5.1 Performance Requirements 15
5.2 Safety Requirements 15
5.3 Security Requirements 15
5.4 Software Quality Attributes 15
5.5 Business Rules 15
Appendix A: Glossary 16

Revision History

Name	Date	Reason For Changes	Version
Rahim, Arish, Hlina, Adam	9/26/25	First Draft of SRS Document	1.0
Rahim, Arish, Hlina, Adam	9/30/25	Iteration of SRS Completed	1.1

1. Introduction

1.1 Purpose

This is version 1.0 of a voting software for use by people who wish to conduct fair and secure voting. It entails the purpose, features, and services this system will provide under certain constraints. This document is intended for users, developers, and stakeholders.

1.2 Document Conventions

This document was created using the IEEE template for System Requirement Documentation.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences:

- Users such as election officials or other authorities who wish to utilize this product to conduct voting.
- Project managers, investors, and other business-related stakeholders who wish to know the specifics of the intentions for this product.
- Testers and verifiers of this product.
- Developers who require technical details to best contribute to the advancement of this product.

1.4 Product Scope

The goal of this system is to conduct fair and accurate elections. Users will have the option of running a plurality or single transferable vote (STV) election and must provide ballot file(s) in CSV format. This system will output results on the screen and create an audit file containing a detailed report of the results and how they were produced.

1.5 References

SRS IEEE Template: https://canvas.umn.edu/courses/518748/files/55197023?module_item_id=15371758
GitHub Repo: https://github.umn.edu/umn-csci-4801-01-F25
Requirements: https://canvas.umn.edu/courses/518748/files/55247399?module_item_id=15377478

2. Overall Description

2.1 Product Perspective

The product is a voting system that is designed to automate the counting of the ballots for a given election process, either plurality or Single Transferable Voting (STV). Ballots are created and

compiled beforehand, then provided to the system in a CSV format. The system is intended to be used by election officials who will use the system to find the winner of an election. The system will incorporate input handling, election processing, and output handling.

2.2 Product Functions

The major functions of the system:

- The CSV file is input to the system through the command line or GUI
- File(s) are verified
- Plurality or STV election is identified
- Election calculation is conducted
- The election winner(s) are determined. In the case of a tie, a coin toss will break the tie
- Election winner and results are processed on the user's screen via the command line or the GUI
- An output file is generated for the user

2.3 User Classes and Characteristics

Programmers and analysts will be using this system during development to develop and test the product. Programmers have the technical background to use the system during the development and validation process to ensure all components of the system are debugged. Once the inner workings of the system are completed and the system is ready for production use, the election official will be using the system to determine the winner of an election. Election officials are the primary end users who will rely on the system for ballot processing and will use the frontend component for a simple application to operate the system.

2.4 Operating Environment

This system is designed to operate on Java (version 17 or later). It will also operate on all CSE lab machines running Ubuntu (version 22 or later).

2.5 Design and Implementation Constraints

The election system is required to process 100,000 ballots in under 5 minutes and must run on CSE lab machines using Java (version 17 or later) and Ubuntu (version 22 or later). Preprocessing of the ballots is not required, assuming the ballots are correct upon input to the system. No security requirements are present; the assumption is that there is no error within the ballots. Both plurality and STV election processes will be used, and a shuffle algorithm will be implemented to eliminate any compilation bias.

2.6 User Documentation

User documentation will be provided for the system via a help menu for usage instructions. A README file will be provided in the GitHub repository for system installation, setup, and usage.

2.7 Assumptions and Dependencies

All ballots given to the system will be given without any errors. Each file will contain at least one valid ballot. One output file will be created. Security of the ballots will be handled outside of the system.

3. External Interface Requirements

3.1 User Interfaces

The system must provide a user interface that will include the command line. The command line interface will collect necessary information from the user, which will include the following:

- Number of seats to fill.
- Election algorithm: Plurality or STV.
- One or more CSV files containing the ballots.
- Shuffle toggle (enable or disable shuffling)

After the algorithm has conducted the election, the results will be displayed on the user interface. This includes:

- Type of election.
- Number of ballots.
- Number of seats.
- Number of candidates.
- Elected candidates.
- Non-elected candidates.
- Win percentages for plurality and order of wins/losses for STV.

If a user needs help with running the election program, the help button will display a GUI explaining how the system works. The help window will explain how the system works to the user in a manner that will take less than 15 minutes to understand.

3.2 Hardware Interfaces

The system will need to run on CSE lab machines.

3.3 Software Interfaces

In order to run the software, Java 17 or later versions must be installed.

3.4 Communications Interfaces

No internet connection is required to run the election.

4. System Features

4.1 Command Line for User Interface

Name	Command Line For User Interface
ID	UC_001
Description	A medium-priority feature intended for programmers and election officials to run the system in an IDE using command-line arguments. The command-line interface will be used to test, shuffle, input ballots, choose the election type, and receive election outputs.
Actors	Programmer, Tester, Election Official
Organizational Benefits	Supports the development and execution of code
Frequency of Use	100% of elections (testing or production) will use this interface
Triggers	Execution of the program in an IDE, terminal, or command line input
Preconditions	Compiled Java Code Input ballots are present Ballots are valid
Postconditions	Election runs using the selected algorithm Election uses ballots given Election results displayed Prompt for audit file generation
Main Course	1. An actor runs a program with arguments 2. System validates arguments and files 3. The system executes the election (See UC_006, UC_007) 4. Election information output to the command line (See UC_004)

	5. Audit file is generated and displayed at the command line (See UC_003)
Alternate Courses	No alternate course
Exceptions	No exceptions

4.2 File Input By Name

Name	File Input By Name
ID	UC_002
Description	A medium-priority feature intended for programmers and election officials to be able to provide one or more files for ballot processing. The system will combine all ballots and ensure headers across the files.
Actors	Programmer, Tester, Election Official
Organizational Benefits	Supports ambiguous file count and flexibility in ballot processing and management.
Frequency of Use	100% of elections (testing or production) will use this interface
Triggers	Actor specifies file(s) in the command line
Preconditions	All files are valid CSV At least one ballot in each file Candidate header consistent across files
Postconditions	All files are aggregated for ballot processing The system determines candidates and ballot processing
Main Course	1. Actor provides valid filename(s) 2. System processes and validates file(s)

	3. The system reads and aggregates the header and ballots 4. Election execution begins (See UC_006, UC_007) 5. The system displays the winner and outputs (audit) information. (See UC_003, UC_004)
Alternate Courses	No alternate course
Exceptions	No exceptions

4.3 Audit File

Name	Audit File
ID	UC_003
Description	Upon completion of an STV election, an audit file will be created. This audit file should contain all necessary information (See Postconditions) about the election.
Actors	System
Organizational Benefits	Allows for easier testing and election verification.
Frequency of Use	50% of elections performed using this system will likely be STV.
Triggers	Completion of an STV election
Preconditions	An STV election has run successfully with results
Postconditions	The audit file is saved as a .txt file The audit file has a name unique to each election The audit file contains:

	● SRS Election ● The date of the election. ● The candidates' names and parties. ● The total number of candidates. ● The number of seats to be filled by the election. ● The Elected list, in the order in which the candidates were added to the list. ● The Non-Elected list, in the order in which candidates were added to the list.
Main Course	1. The system completes an STV election (See UC_006) 2. The audit file is compiled with all the necessary information (See postconditions) 3. The audit file is saved as a human-readable .txt file
Alternate Courses	No alternate course
Exceptions	No exceptions

4.4 Display Results

Name	Display Results
ID	UC_004
Description	After an election is completed, a results screen will be visible to the user. This results screen should display all relevant information about the results of the election (See Postconditions).
Actors	System
Organizational Benefits	The results of the election will be visible to the user running the program, allowing for ease of use as compared to requiring the user to find and read the audit file.

Frequency of Use	100% of the time this system is used for an election, this result screen will be displayed.
Triggers	Completion of any election
Preconditions	An election has run successfully with results.
Postconditions	The screen displays the results. Information displayed includes: ● Type of election ● Number of ballots processed ● Number of seats ● Number of candidates ● Percentage of votes received in the case of a plurality election ● The order of winners and losers in the case of an STV election
Main Course	1. The system runs an election (See UC_006, UC_007) 2. The system completes an election 3. The system compiles the relevant information from the result of the election 4. The system prints all of the compiled information onto the screen
Alternate Courses	No alternate course
Exceptions	No exceptions

4.5 Tie Handling

Name	Tie Handling
ID	UC_005
Description	Ties will be handled by the system as they occur in either the STV or plurality election processes. The result should be the correct

	number of winners and losers being selected based on the number of seats.
Actors	Election algorithm
Organizational Benefits	The elections will have fair results with as many winners as there are seats available.
Frequency of Use	As often as ties occur.
Triggers	A tie is encountered by either the plurality or the STV election algorithms.
Preconditions	The number of seats available must be known. The type of election is identified. For STV, the order in which their ballots were received must be known.
Postconditions	The number of winners chosen from the tie matches the number of seats available. The winners are chosen in a fair way that is statistically random.
Main Course	1. The type of election is identified (See UC_001) 2. An election is started (See UC_006, UC_007) 3. An election is in progress (See AC2) 4. The winners are chosen (See AC1, AC3)
Alternate Courses	AC1 The election is a plurality, and there are fewer winners than seats 1. Assign a random number to each tied candidate within a given range. 2. Choose a number randomly within that range. 3. Choose the candidate closest to that number that doesn't exceed that number, and assign them as a winner. 4. If there are more seats to fill, repeat AC1 5. Return to Main Course step 3. (See EX1)

	AC2 The election is STV, and there are losers tied at the end of a round 1. Assign a random number to each tied candidate within a given range. 2. Choose a number randomly within that range. 3. Choose the candidate closest to that number that doesn't exceed that number, and assign them as the loser of the current round. 4. Return to Main Course step 2. AC3 The election is STV, and all candidates are winners or losers, but there are still open seats 1. The last loser is promoted to a winner. 2. If there are more seats to be filled, repeat AC3 3. Return to Main Course step 3. (See EX2)
Exceptions	EX1 There are more seats to fill in a plurality election with ties remaining 1. Return to AC1 step 1 EX2 There are still seats open at the end of an STV election 1. Return to AC3 step 1

4.6 STV Algorithm

Name	STV Algorithm
ID	UC_006
Description	When the user provides the necessary inputs, such as CSV files (for the ballots), the number of seats, and the shuffle toggle, the STV algorithm will run (if selected). The algorithm calculates Droop, creates a list of the elected and non-elected candidates, outputs an audit report (text file), and displays the results on the user interface.

Actors	Election officials and testers.
Organizational Benefits	The system will produce a reliable algorithm that ensures fairness and randomization (shuffle) with an audit trail.
Frequency of Use	The STV algorithm should complete in less than 5 minutes with up to 100,000 ballots.
Triggers	Through the user interface, the user must select STV as a type and provide the required inputs.
Preconditions	Users must be running the code on CSE lab machines. CSV files must be in the same directory and formatted properly.
Postconditions	Elected and Non-Elected lists are displayed and are accurate. An audit report text file includes the necessary information. The user interface will display a quick summary of the results.
Main Course	1. The user chooses STV. 2. If shuffle is on, the ballots will be shuffled. (See UC_008) 3. STV calculates Droop. 4. Going one at a time, the ballots are piled up, and any candidate reaching droop is immediately elected. Their ballots are removed from the election. The following ballots will consider the next person in their ranking. 5. STV will eliminate candidates with the fewest number of votes. 6. Continue the algorithm until all ballots have been accounted for to develop elected and non-elected lists. 7. The system will output results and create an audit report. (See UC_003, UC_004)
Alternate Courses	The STV algorithm will not change based on shuffle or the number of input files.
Exceptions	None.

4.7 Plurality Algorithm

Name	Plurality Algorithm
ID	UC_007
Description	When the user provides the necessary inputs, such as ballot CSV files and the number of seats, the plurality algorithm will run (if selected). This algorithm will assign each ballot to only one candidate, and the winner(s) will be those who received the most votes. Ties will be handled with a coin toss, and the results will be displayed on the user interface.
Actors	Election officials and testers.
Organizational Benefits	The system will produce a reliable algorithm that ensures fairness and randomization (shuffle) with an audit trail.
Frequency of Use	The Plurality algorithm should complete in less than 5 minutes with up to 100,000 ballots.
Triggers	Through the user interface, the user must select Plurality as a type and provide the required inputs.
Preconditions	Users must be running the code on CSE lab machines. CSV files must be in the same directory and formatted properly.
Postconditions	- Winner(s) will be identified. - UI will show the results.
Main Course	1. The UI will prompt for the number of seats, algorithm type, and CSV files (See UC_001) 2. The program will read in the ballots and assign each one to a candidate. (See UC_002) 3. The total number of votes will decide the winner. 4. Random coin tosses will determine the winner in case of a tie. (See UC_005)

	5. UI will display the results at the end. (See UC_004)
Alternate Courses	None.
Exceptions	None.

4.8 Shuffle

Name	Shuffle
ID	UC_008
Description	During an official election, shuffle will randomize the order of the ballots once before STV (see UC_006) to ensure the fairness of STV voting.
Actors	System
Organizational Benefits	By using shuffling, the system will ensure that the order in which ballots are received will not disproportionately affect candidates' chances of being elected to a seat.
Frequency of Use	Every time an official election is being conducted, shuffle mode should be automatically turned on and should take place during each round of ballot evaluation.
Triggers	Through the user interface, when the user selects STV, the system will automatically utilize shuffling.
Preconditions	Users must be running the code on CSE lab machines. CSV files must be in the same directory and formatted properly.
Postconditions	Ballots will not be in the same order in which they were received and will also be subject to change between rounds of ranking evaluation.
Main Course	1. The user chooses STV to conduct an official election. (See UC_001)

	2. Shuffle mode will automatically be activated and randomize the order of the ballots. 3. STV algorithm will take place (See UC_006)
Alternate Courses	AC1 The user is a tester/developer who will thus have the option of conducting an unofficial election for testing purposes. 1. If a tester or developer chooses to conduct an election for testing, shuffle mode may be turned off, so the system will not randomize ballots. 2. The system will conduct the STV algorithm (see UC_006)
Exceptions	None.

4.9 Help Window

Name	Help Window
ID	UC_009
Description	When a user would like help to interact with the program before an election starts, a user can request help through a help window that is accessible through the command line (see UC_001).
Actors	Election officials and testers.
Organizational Benefits	The system will provide users with clarification on how to properly interact with the system.
Frequency of Use	The help window can be accessed an unlimited number of times at the user's discretion.
Triggers	When a user prompts the help window by typing "help" in the text interface, the help window will be launched.
Preconditions	An election must not be actively taking place.

Postconditions	The user will have received prompting questions to help them utilize the system for their needs.
Main Course	1. The UI will ask the user what kind of election they would like to conduct, STV or plurality. They will also have the option of typing “help” to receive help. 2. If the user enters “help”, the UI will provide a brief explanation of STV voting and plurality voting, before asking again which they would like to conduct.
Alternate Courses	None.
Exceptions	None.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The program must complete an election within 5 minutes.

5.2 Safety Requirements

The system will not modify any ballot files given to it to prevent ballot data from being damaged or lost. The audit files will be output with unique names to ensure election data is not lost.

5.3 Security Requirements

The system will not allow any input while the election is running to prevent election tampering.

5.4 Software Quality Attributes

The software will allow for testing to be done using the command line. Testing will allow for the monitoring of election results and inputting test files for the election. Training for the interface should take no longer than 15 minutes to learn. The system will produce the correct election result 100% of the time. The software must be able to run on any machine meeting system requirements (see 3.2 and 3.3).

5.5 Business Rules

Testers may use command-line arguments. Users may never use command-line arguments. Users, programmers, testers, and administrators should be able to enable and disable shuffle.

Appendix A: Glossary

UI (User Interface)
How the user interacts with the program

Plurality
A voting system in which the winner is determined by having more votes than the other candidates

Droop
The number of votes required to be declared a winner in an STV election

CSV (Comma Separated Values)
Plain text file that contains information separated by commas. Read by the program to tabulate ballots.

IDE (Integrated Development Environment)
Software applications are used to write, debug, compile, and/or execute code.

STV (Single Transferable Voting)
A method of voting that allows voters to rank candidates and transfer votes if their preferred candidate is eliminated.

GUI (Graphical user interface)
This term is used to describe the parts of the software system that the user interacts with

GitHub
Version control platform for developers to effectively collaborate on software development

CSE Lab Machine (College of Science and Engineering) Lab Machine
Computers that can be accessed in person on the University of Minnesota Campus or remotely through an SSH connection

README file
Mark down a text file with background information about the system